

Основы тестирования IT проектов

А работает ли оно вообще?

Сегодня мы попробуем посмотреть на цифровой проект глазами тестировщика.

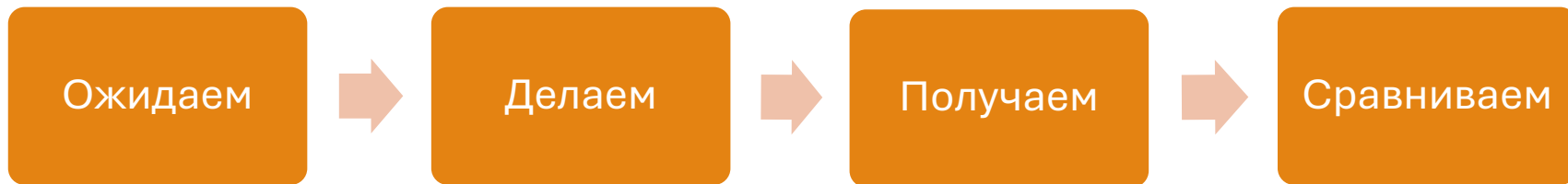
Представьте, что вам дали приложение, сайт или чат-бот и сказали:
"Проверьте, всё ли здесь работает".

Что вы будете делать? Просто нажимать на кнопки? Искать ошибки?
Пытаться его сломать?

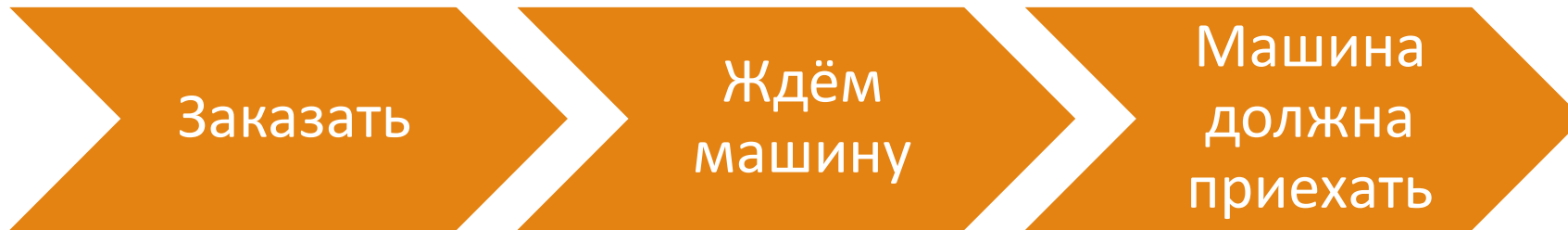
Тестирование — это гораздо более систематический процесс. Сегодня мы попробуем разобраться, как обычный пользователь превращается в тестировщика».

Что такое тестирование?

Тестирование — это проверка цифрового продукта, чтобы понять, работает ли он так, как ожидается.



Обычный заказ такси



Что именно мы тестируем?

ФУНКЦИИ

- Что умеет делать продукт?

ИНТЕРФЕЙС

- Как это выглядит и воспринимается?

УДОБСТВО

- Насколько легко этим пользоваться?

Что такое функция?

Функции информационной системы — это основные действия (элементарные операции), которые система выполняет для работы с информацией и обеспечения пользователей нужными данными.

Примеры:

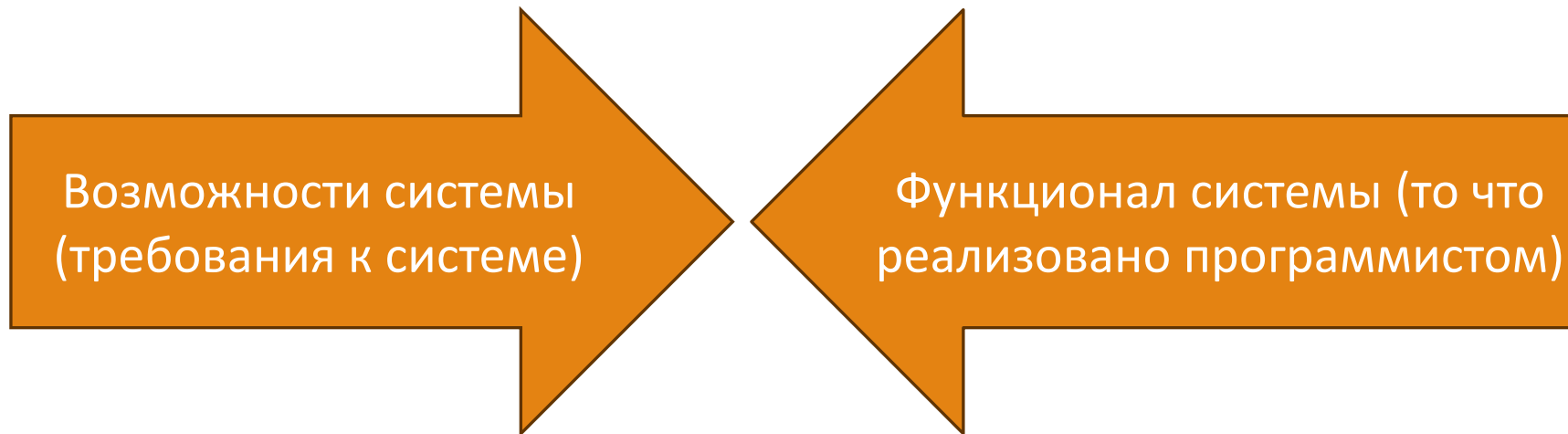
- ☐ входить;
- ☐ зарегистрироваться;
- ☐ найти;
- ☐ отправить;
- ☐ получить информацию;
- ☐ выбрать;
- ☐ вернуться назад.

Что такое требование?

Требование = возможности системы (что система должна делать).

Пример:

- ☐ Пользователь должен иметь возможность выполнить поиск.
- ☐ Система должна показать результат поиска.
- ☐ Пользователь должен иметь возможность вернуться назад.



Требование Тест

Требование	Что проверяем
Можно выполнить поиск	Работает ли поиск?
Можно получить результат	Появляется ли результат?
Можно вернуться назад	Работает ли кнопка?

Что такое тестовый сценарий?

Тестовый сценарий — это инструкция, по которой мы выполняем проверку

1. Открыть проект.
2. Запустить бота.
3. Ввести запрос.
4. Нажать кнопку.
5. Проверить результат.

Хороший и плохой тест

 Плохой

Проверить поиск.

 Хороший

1. Открыть проект.
2. Выбрать поиск.
3. Ввести «спорт».
4. Нажать «Найти».
5. Проверить результат.

Ожидаемый результат

Что должно произойти?

Пример:

Действие: ввести «спорт».

Ожидаем: появляются подходящие результаты.

Фактический результат

Что произошло на самом деле?

Пример:

Ожидали: список элективов.

Получили: сообщение «Ничего не найдено».

PASS или FAIL?

-  PASS Тест пройден.
-  FAIL Тест не пройден.
-  PARTIAL Тест пройден частично.

Баг — это не «мне не нравится»



Мне не нравится дизайн.



Это ещё не обязательно баг.

Как выглядит хорошее описание бага?

 Плохо:

Поиск не работает.

 Хорошо:

После ввода ключевого слова и нажатия кнопки «Поиск» результаты не отображаются.

Функциональное тестирование

Проверка логики и операций с данными

Функциональное тестирование отвечает на вопрос: делает ли система то, что должна делать? Оно проверяет внутреннюю логику продукта.

Проверяем:

- ☐ обработку ввода пользователя (*поиск, ввод текста*);
- ☐ выполнение расчётов и преобразований данных;
- ☐ сохранение информации (*регистрация, отправка сообщения*);
- ☐ получение результата от сервера;
- ☐ работу основных функций (*авторизация, поиск, добавление в корзину*);
- ☐ реакцию на действия пользователя (*нажатие кнопок, переходы между экранами*).

Тестирование интерфейса

Проверяем:

 Видно?

 Понятно?

 Понятна цель?

 Удобно?

Позитивное тестирование

Позитивное тестирование: «Сценарий по правилам». Проверка работы системы при корректных входных данных.

Позитивное тестирование — это проверка того, как система ведёт себя в идеальных условиях, когда пользователь делает всё правильно.

Примеры позитивного сценария:

- ❑ **Поиск:** Вводим корректный запрос («программирование») → ожидаем увидеть релевантные результаты.
- ❑ **Регистрация:** Заполняем все поля валидными данными (e-mail, пароль) → ожидаем успешную регистрацию.
- ❑ **Операция:** Нажимаем кнопку «Отправить» после заполнения формы → ожидаем отправку сообщения без ошибок.

Цель: Убедиться, что основные функции продукта работают так, как задумано разработчиками.

Негативное тестирование

А что будет, если пользователь сделает что-то неожиданное?

Попробуем:

- ☐ ничего не ввести;
- ☐ ввести неправильный запрос;
- ☐ сделать опечатку;
- ☐ ввести очень длинный текст;
- ☐ ввести цифры;
- ☐ использовать символы;
- ☐ быстро выполнить несколько действий.

Чек-лист: план действий тестировщика

Чек-лист — это структурированный список всех проверок, которые необходимо выполнить при тестировании продукта. Он помогает избежать хаотичного тестирования и гарантирует, что ключевые функции будут проверены.

Цель: Обеспечить системный подход к проверке качества продукта.

Примеры пунктов чек-листа:

- ☐ **Запуск проекта:** Открывается ли проект без ошибок? Работает ли он стабильно?
- ☐ **Навигация:** Все ли кнопки работают? Можно ли вернуться назад после выполнения действия?
- ☐ **Функциональные блоки:** Поиск работает корректно? Результаты поиска релевантны запросу?
- ☐ **Интерфейс:** Элементы управления видны и понятны? Шрифты читаемы на разных устройствах?
- ☐ **Юзабилити:** Интуитивен ли процесс использования? Нет ли лишних или непонятных шагов?

От чек-листа к таблице

ЧЕК-ЛИСТ

- Проверить поиск.

ТЕСТ

- Действия
Ожидаемый результат
Фактический результат
Статус

Таблица тестирования

Поле	Значение
№	1
Раздел	Навигация
Объект проверки	Кнопка «Помощь»
Действия (сценарий тестирования)	1. Открыть главное меню. 2. Найти «Помощь». 3. Нажать кнопку
Ожидаемый результат	Открывается раздел помощи
Фактический результат	Ничего не происходит
Статус	 Не пройден
Тип проблемы	Функциональный баг
Важность	Высокая
Скриншот	Скриншот 1
Рекомендации	Проверить работу кнопки

Как определить важность проблемы?

Приоритеты дефектов: как расставлять точки над «i». Не все баги одинаково опасны

Для эффективной работы тестировщика важно не только найти проблему, но и правильно оценить её критичность. Это помогает разработчикам понять, в каком порядке исправлять ошибки.

Приоритет	Описание	Пример из проекта
 Высокий (Critical)	Основная функция продукта полностью недоступна или работает некорректно. Пользователь не может выполнить ключевую задачу. Система падает (<i>crash</i>).	Поиск по ключевым словам всегда возвращает пустой список («Ничего не найдено»). Кнопка «Отправить» не отправляет сообщение — оно просто исчезает.
 Средний (Major/Medium)	Функция работает, но использование затруднено. Ошибка вызывает неудобство, потерю данных или требует лишних действий от пользователя.	В форме поиска поле «Название курса» слишком узкое, текст обрезается, приходится вводить вслепую. После отправки сообщения страница перезагружается вместо плавного перехода.
 Низкий (Minor/Low)	Косметические недостатки интерфейса, мелкие недочёты, которые не мешают основной работе системы.	Цвет текста кнопки сливается с фоном, но кнопку всё равно можно нажать. На странице есть лишний пробел между элементами.

Тестировщик ищет не только ошибки

Полная картина: баланс между критикой и похвалой

Как составить полезный отчёт о тестировании?

Хороший результат тестирования — это всесторонний анализ продукта:

-  **Что работает хорошо?** *(Положительные моменты)*
-  **Что работает неправильно?** *(Функциональные дефекты / баги)*
-  **Что неудобно?** *(Проблемы интерфейса/юзабилити)*
-  **Что можно улучшить?** *(Предложения по развитию)*

Цель: Предоставить разработчикам объективную обратную связь, а не просто список жалоб.

Тестировщик ищет не только ошибки

Пример:

Категория	Результаты теста
 Хорошо	Поиск выдаёт релевантные результаты за доли секунды.
 Неправильно	При вводе длинного запроса система зависает и показывает ошибку сервера.
 Неудобно	Кнопка «Отправить вопрос» находится внизу экрана, её трудно найти без прокрутки.
 Улучшить	Добавить возможность сортировки результатов поиска по дате добавления.

Теперь Вы — тестировщики

Алгоритм работы:

1. Изучить проект



2. Найти функции



3. Составить чек-лист проверки



4. Выполнить тесты



5. Заполнить таблицу

Ваши 5 вопросов

Перед каждой проверкой спросите себя:

1. Что я проверяю?
2. Что должно произойти?
3. Что я сделал?
4. Что произошло?
5. Совпали ли результат и ожидание?